condition (so gets will return nil, for example). If ARGV is initially empty, ARGF will read from standard input.

You can get to the name of the file currently being read from using ARGF.filename, and you can get the current File object as ARGF.file. ARGF keeps track of the total number of lines read in ARGF.lineno—if you need the line number in the current file, use ARGV.file.lineno. Here's a program that uses this information:

```
while (line = gets)
  printf "%d: %10s[%d] %s", ARGF.lineno, ARGF.filename, ARGF.file.lineno, line
end
```

If we run it, passing a couple of file names, it will copy the contents of those files.

```
$ ruby copy.rb testfile otherfile
1:   testfile[1] This is line one
2:   testfile[2] This is line two
3:   testfile[3] This is line three
4:   testfile[4] And so on...
5:  otherfile[1] ANOTHER LINE ONE
6:  otherfile[2] AND ANOTHER LINE TWO
7:  otherfile[3] AND FINALLY THE LAST LINE
```

In-Place Editing of ARGF Files

In-place editing is a hack inherited from Perl. It allows you to alter the contents of files passed in on the command line, retaining a backup copy of the original contents.

To turn on in-place editing, give Ruby the file extension to use for the backup file, either with the -i [_ext_] command-line option, or by calling ARGF.inplace_mode=_ext_ in your code.

Now, as your code reads through each file given on the command line, Ruby will rename the original file by appending the backup extension. It will then create a new file with the original name, and open it for writing on standard output. This all means that if you code a program such as this:

```
while (line = gets)
  puts line.chomp.reverse
end
```

and you invoked it using

```
$ ruby -i.bak reverse.rb testfile otherfile
```

You'd find that testfile and otherfile would now have reversed lines, and that the original files would be available in testfile.bak and otherfile.bak.

For finer control over the I/O to these files, you can use the methods provided by ARGF. They're rarely used, so rather than document them here, we'll refer you to the online documentation.

Option Parsing

It's quite handy that Ruby packages up all the options into the ARGV array. If you have a complex script, and you want your script to use conventional patterns of options, where there's something like -a true --database sqlite, then the ARGV array isn't quite enough, you'd